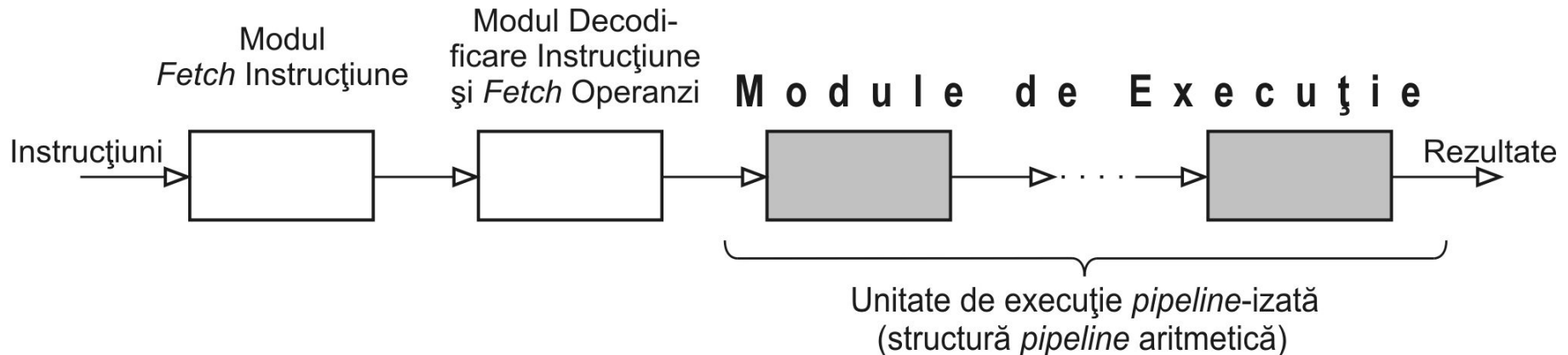


Structuri *pipeline* aritmetice

1. Considerații generale

Conceptul *pipeline*-izării se aplică și la nivelul operației aritmetico-logice vizând astfel structura unităților de execuție din cadrul procesorului. Structurile *pipeline* aritmetice sunt utilizate pentru implementarea funcțiilor aritmetice complexe cum ar fi: adunarea/scăderea în virgulă flotantă, înmulțirea/împărțirea în virgulă fixă și respectiv flotantă, etc. Se obțin astfel structuri *pipeline* aritmetice care se inserează în *pipeline*-ul de la nivelul instrucțiunii, atunci când o instrucțiune complexă ajunge în faza de execuție:

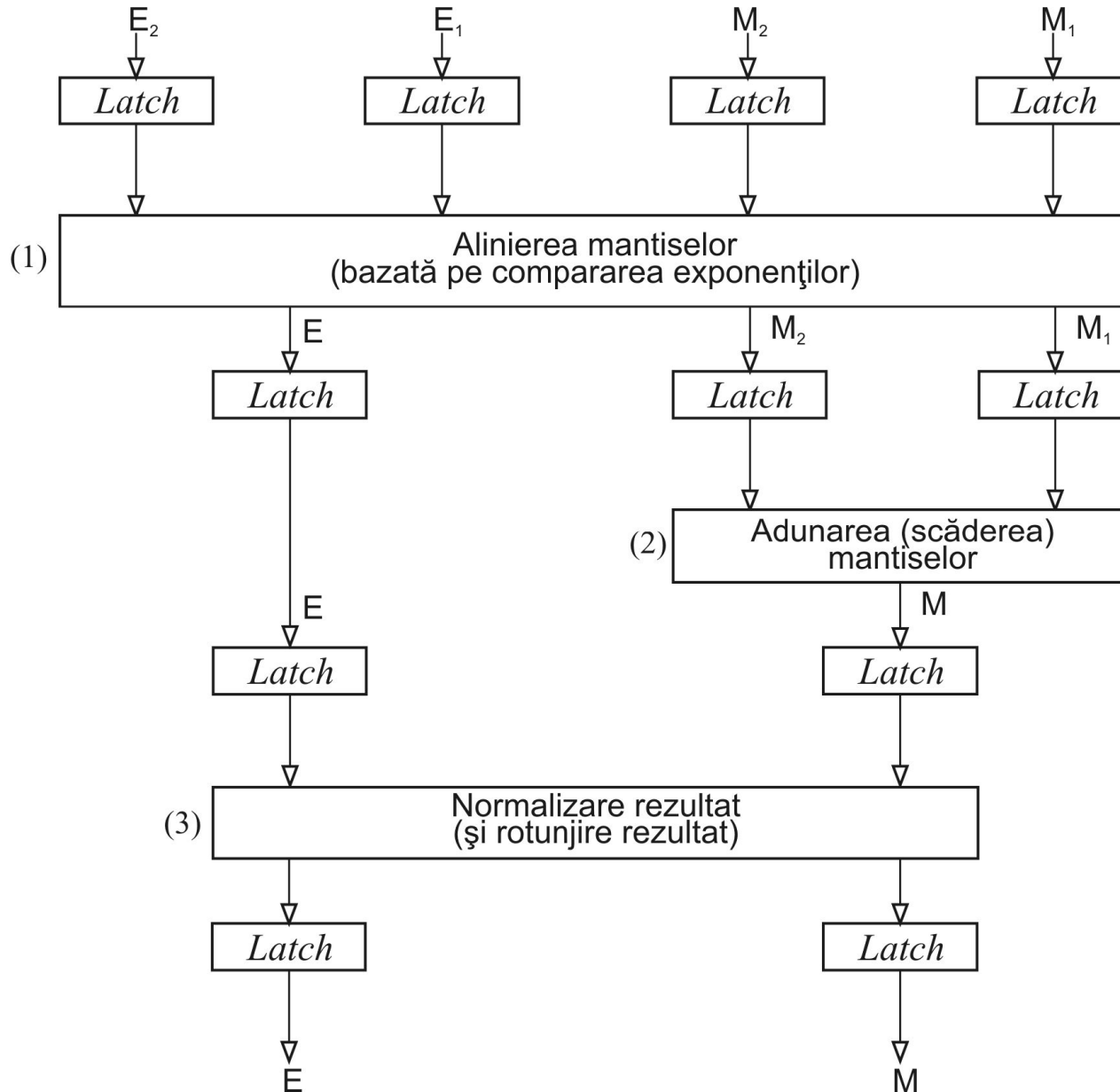


O instrucțiune tipică în virgulă flotantă ar fi:

FADD FR1,FR2,FR3 ; FR1 = FR2 + FR3

Structuri *pipeline* aritmetice

1. Considerații generale



Exemplu:

$M_1=0,1110$

$E_1=2^4$

$M_2=0,1000$

$E_2=2^2$

$M_1=0,1110$

$M_2=0,0010$

$E=2^4$

$M=1,0000$

$E=2^4$

$M=0,1000$

$E=2^5$

Structură *pipeline* pe 3 nivele pentru adunarea (scăderea) în virgulă flotantă

Structuri *pipeline* aritmetice

1. Considerații generale

Aceeași unitate de adunare în virgulă flotantă ar putea fi implementată într-o structură *pipeline* pe 5 nivele. Funcția celor 5 module din cascada *pipeline* ar fi:

- scade exponenții $E1$, $E2$, și generează diferența $E1-E2$
- interschimbă mantisele $M1$ și $M2$, dacă diferența $E1-E2 < 0$ și pozitivează astfel diferența $E1-E2$; dacă diferența $E1-E2 > 0$ acest modul nu execută nicio operație.
- deplasează mantisa $M2$ la dreapta cu $E1-E2$ poziții (alinie mantise)
- adună mantisele $M1$ și $M2$ generând mantisa rezultat M . (scade mantisele pentru operația de scădere).
- normalizează rezultatul. Dacă mantisa este mai mică decât $0,5_{10}$ (nu are 1 pe prima poziție după virgulă), mantisa se deplasează la stânga până când prima cifră după virgulă devine 1 și se scade numărul de deplasări din valoarea exponentului. Dacă mantisa M este mai mare ca 1, este deplasată cu o poziție la dreapta și se adună 1 la exponent. Dacă mantisa rezultat este normalizată nu se execută nici o operație.

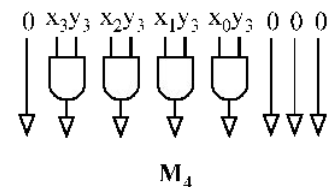
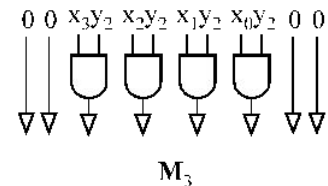
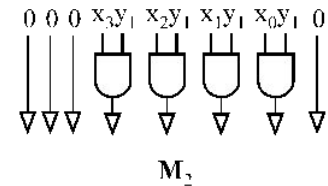
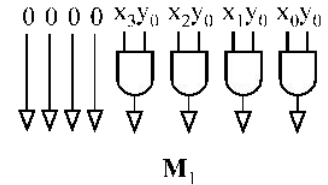
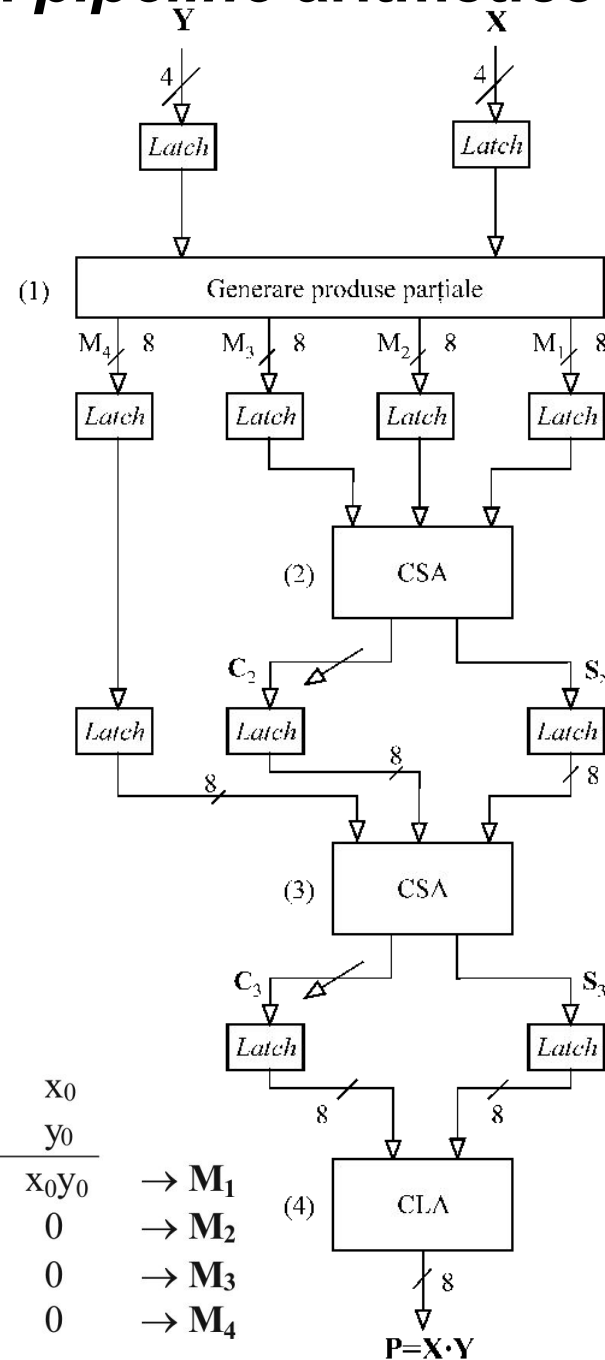
Înmulțirea în virgulă flotantă poate fi implementată într-o structură *pipeline* cu 5 nivele:

- adună exponenții $E1$ și $E2$.
- înmulțește mantisele $M1$ și $M2$.
- normalizează rezultatul (dacă este necesar).
- rotunjește mantisa rezultat reducând-o la lungime simplă.
- renormalizează mantisa rezultat dacă este necesar (rotunjirea poate necesita adunarea unui 1 pe ponderea cea mai puțin semnificativă a mantisei ceea ce impune renormalizarea).

Structuri *pipeline* aritmetice

1. Considerații generale

Structură *pipeline* de înmulțire
în virgulă fixă cu
"carry save adders" (CSA)



				x ₃	x ₂	x ₁	x ₀	
				y ₃	y ₂	y ₁	y ₀	
0	0	0	0	x ₃ y ₀	x ₂ y ₀	x ₁ y ₀	x ₀ y ₀	→ M ₁
0	0	0	x ₃ y ₁	x ₂ y ₁	x ₁ y ₁	x ₀ y ₁	0	→ M ₂
0	0	x ₃ y ₂	x ₂ y ₂	x ₁ y ₂	x ₀ y ₂	0	0	→ M ₃
0	x ₃ y ₃	x ₂ y ₃	x ₁ y ₃	x ₀ y ₃	0	0	0	→ M ₄

Structuri *pipeline* aritmetice

1. Considerații generale

Modulul (1) generează produsele parțiale $\mathbf{M}_1$, $\mathbf{M}_2$, $\mathbf{M}_3$ și $\mathbf{M}_4$:

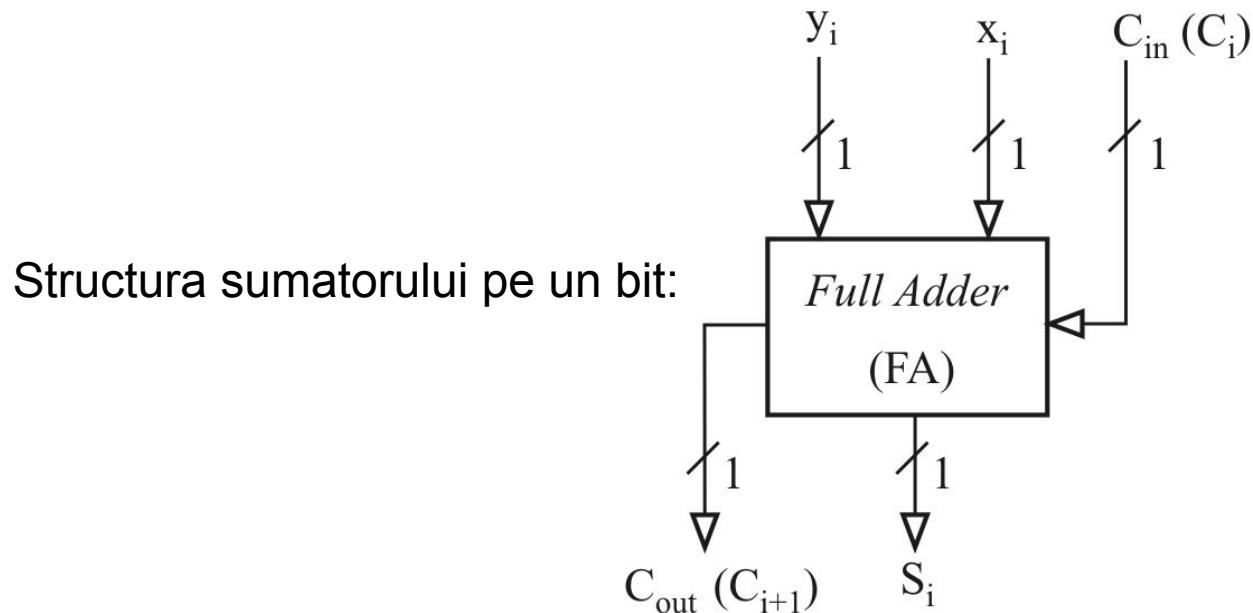
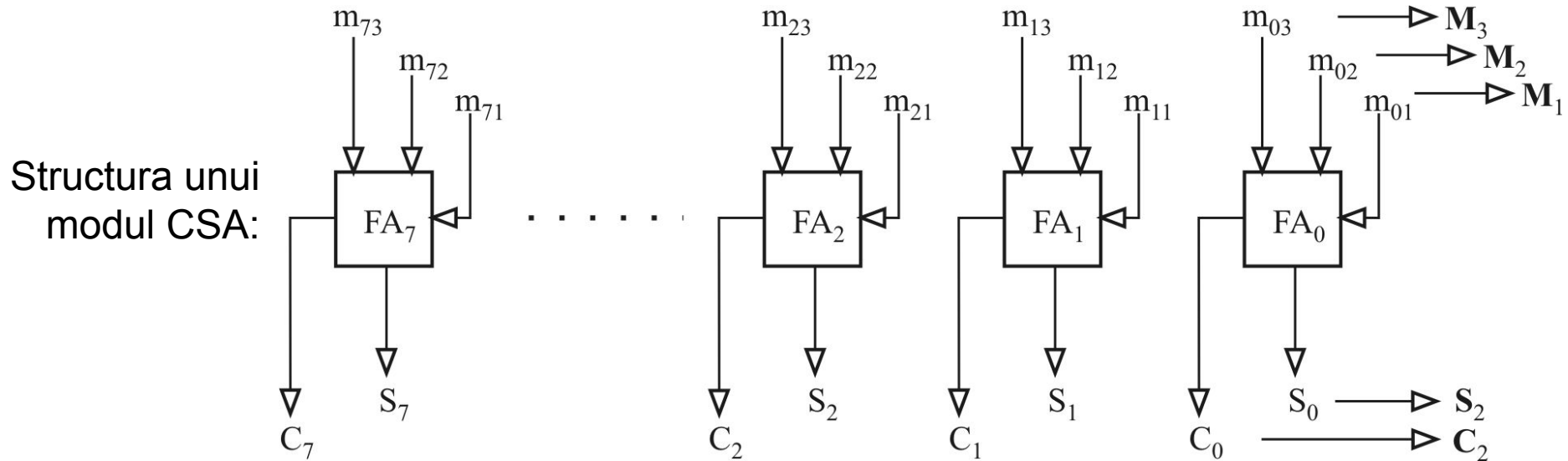
				x_3	x_2	x_1	x_0	
				y_3	y_2	y_1	y_0	
0	0	0	0	x_3y_0	x_2y_0	x_1y_0	x_0y_0	$\rightarrow \mathbf{M}_1$
0	0	0	x_3y_1	x_2y_1	x_1y_1	x_0y_1	0	$\rightarrow \mathbf{M}_2$
0	0	x_3y_2	x_2y_2	x_1y_2	x_0y_2	0	0	$\rightarrow \mathbf{M}_3$
0	x_3y_3	x_2y_3	x_1y_3	x_0y_3	0	0	0	$\rightarrow \mathbf{M}_4$

Produsele parțiale $\mathbf{M}_1$, $\mathbf{M}_2$, $\mathbf{M}_3$ și $\mathbf{M}_4$ sunt însumate apoi cu ajutorul modulelor (2) și (3); sunt două module CSA – *Carry Save Adder* cu structuri identice.

Modulul (4) este un sumator pe 8 biți cu generare directă a biților de transport (CLA – *Carry Lookahead Adder*)

Structuri *pipeline* aritmetice

1. Considerații generale



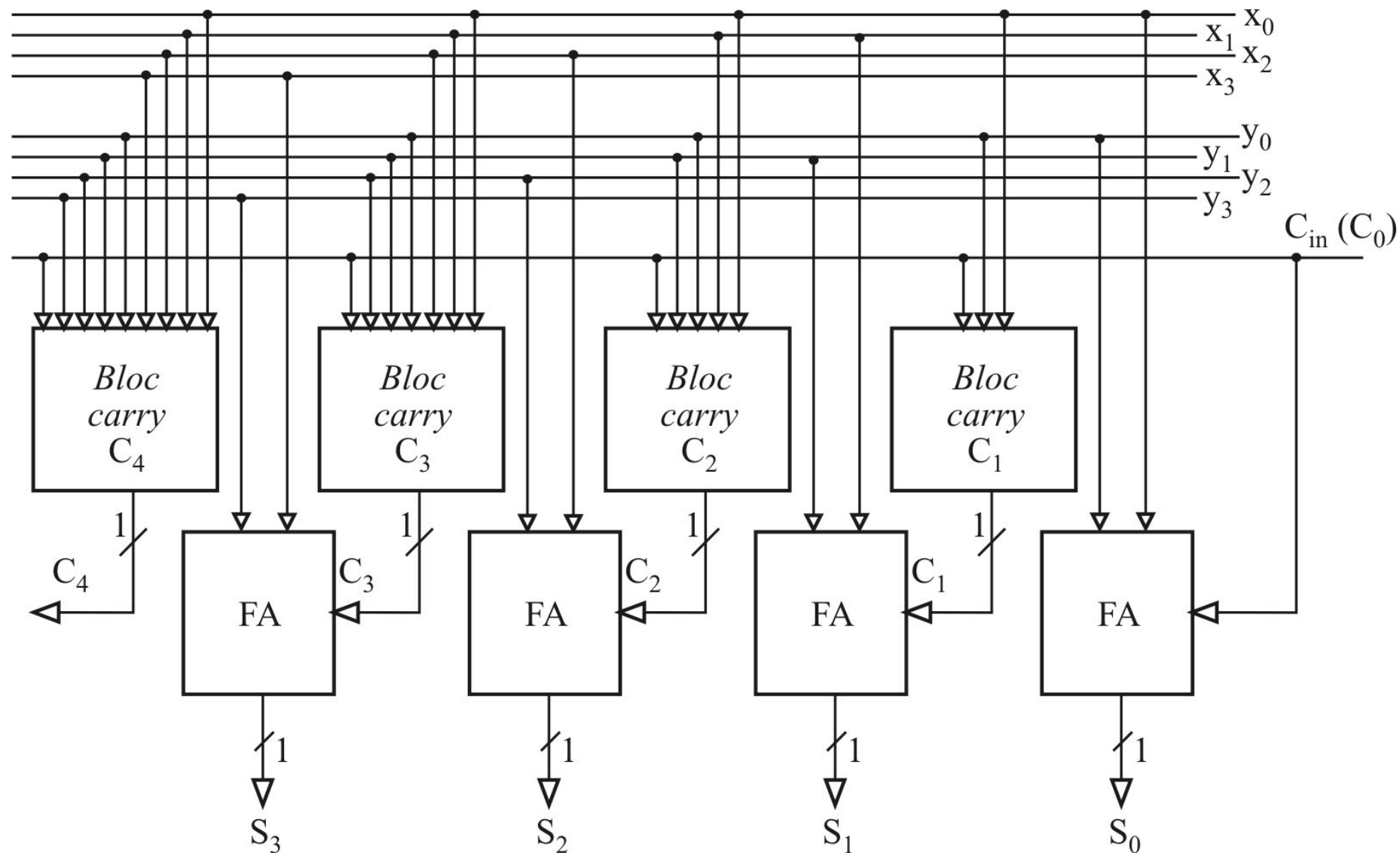
$$S_i = x_i \oplus y_i \oplus C_i$$

$$C_{i+1} = x_i y_i + x_i C_i + y_i C_i$$

Structuri *pipeline* aritmetice

1. Considerații generale

Structura modului CLA:



Structuri *pipeline* aritmetice

1. Considerații generale

Funcționarea modului CLA:

Funcția fiecărui bloc de *carry* se obține pornind de la expresia:

$$C_{i+1} = x_i y_i + C_i(x_i + y_i)$$

Particularizând: $C_1 = x_0 y_0 + C_0(x_0 + y_0)$

$$C_2 = x_1 y_1 + C_1(x_1 + y_1) = x_1 y_1 + [x_0 y_0 + C_0(x_0 + y_0)](x_1 + y_1)$$

.....

Pentru simplificarea expresiilor C_i , se utilizează adesea notațiile:

$g_i = x_i y_i$;transport generat în rangul i , adică C_{i+1} va fi 1 atunci când g_i este 1

$p_i = x_i + y_i$;condiție de propagare a transportului din rangul anterior (C_i) spre

C_{i+1} (C_{i+1} va fi 1 atunci când p_i și C_i sunt simultan 1).

Cu aceste notații: $C_{i+1} = g_i + p_i C_i$

Particularizând: $C_1 = g_0 + C_0 p_0$

$$C_2 = g_1 + p_1 g_0 + p_1 p_0 C_0$$

$$C_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 C_0$$

$$C_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 C_0$$

.....

Aceste ultime relații sunt implementate în blocurile de *carry*

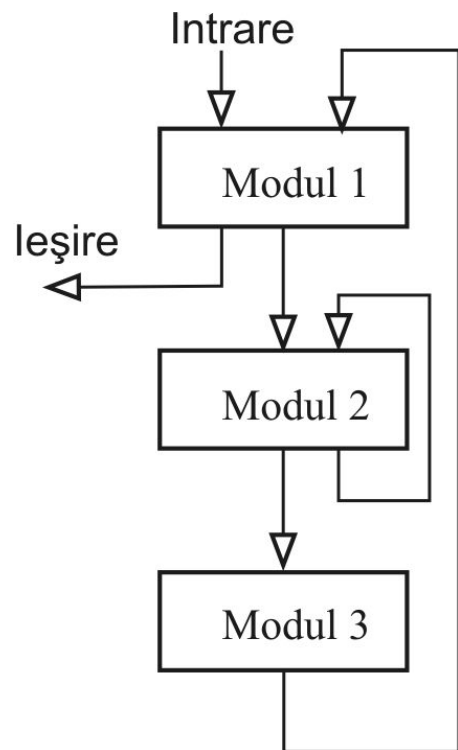
Structuri *pipeline* aritmetice

2. Controlul structurilor *pipeline*

Controlul secvenței de *task*-uri aplicate pe intrarea unei structuri *pipeline* vizează:

- maximizarea performanțelor structurii (găsirea latenței minime dintre 2 *task*-uri succesive)
- evitarea coliziunilor (situațiile în care două *task*-uri anterior inițiate revendică același modul *pipeline* în același moment de timp)

A. Tabela de rezervare



a) Structură *pipeline* statică

	t_0	t_1	t_2	t_3	t_4
Modul 1	x				x
Modul 2		x	x		
Modul 3				x	

b) Tabela de rezervare aferentă

Structuri *pipeline* aritmetice

3. Controlul structurilor statice

A. Lista latențelor interzise

Este o simplă listă de întregi care corespund latențelor nepermise.

Exemplu:

Tabela de rezervare:

	t_0	t_1	t_2	t_3	t_4
Modul 1	x				x
Modul 2		x	x		
Modul 3				x	

are lista latențelor interzise formată din 3 întregi: **$F=(4, 1, 0)$** .

Fiecare întreg din listă corespunde distanței (pe orizontală) dintre două semne x situate pe același rând în tabelă.

Lista se obține evaluând toate distanțele pentru toate rândurile.

Evident contează doar rândurile pe care există mai mult de un semn x.

Latența 0 este întotdeauna nepermisă pentru structurile statice deoarece corespunde inițierii simultane a 2 *task*-uri pe intrarea structurii; ar rezulta coliziuni în fiecare modul.

Structuri *pipeline* aritmetice

3. Controlul structurilor statice

B. Vectorul de coliziune

Un vector de coliziune este format dintr-un șir de cifre binare de lungime **$N+1$** , unde **N** reprezintă cel mai mare întreg (latență) din lista latențelor interzise.

Pentru lista latențelor interzise $F=(4, 1, 0)$, vectorul de coliziune va fi:

$$\begin{array}{cccccc} & c_4 & c_3 & c_2 & c_1 & c_0 \\ C = (& 1 & 0 & 0 & 1 & 1) \\ & 4 & 3 & 2 & 1 & 0 & \leftarrow \text{latența} \end{array}$$

Bitul c_i din vector va fi:

- 1 dacă i reprezintă o latență interzisă (latență existentă în lista F)
- 0 dacă i reprezintă o latență permisă (latență inexistentă în lista F)

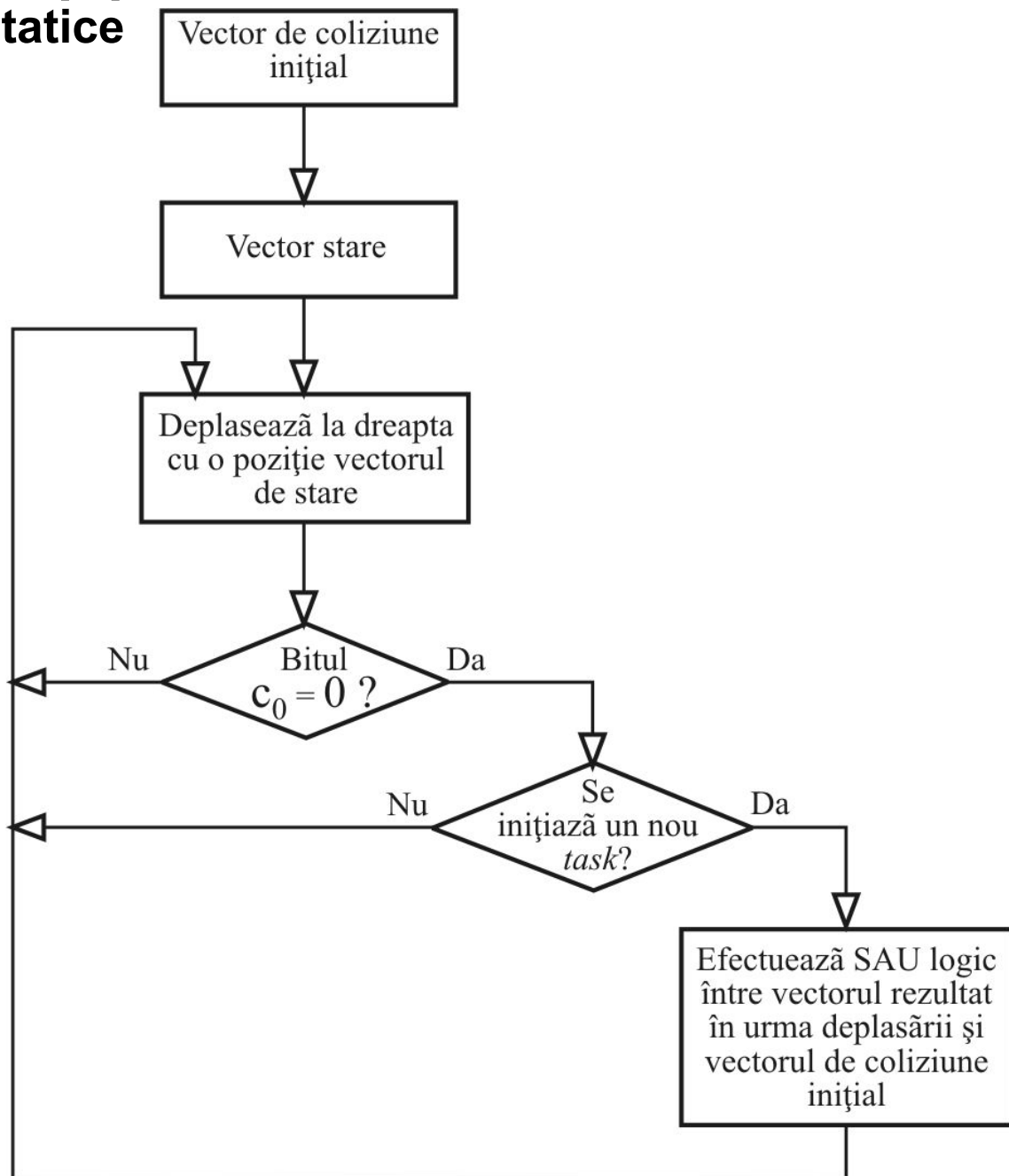
Să remarcăm faptul că vectorul de coliziune $C=10011$ permite latențele 2 și 3, dar nu permite latențele 0, 1 și 4.

Structuri *pipeline* aritmetice

3. Controlul structurilor statice

C. Organigrama stărilor

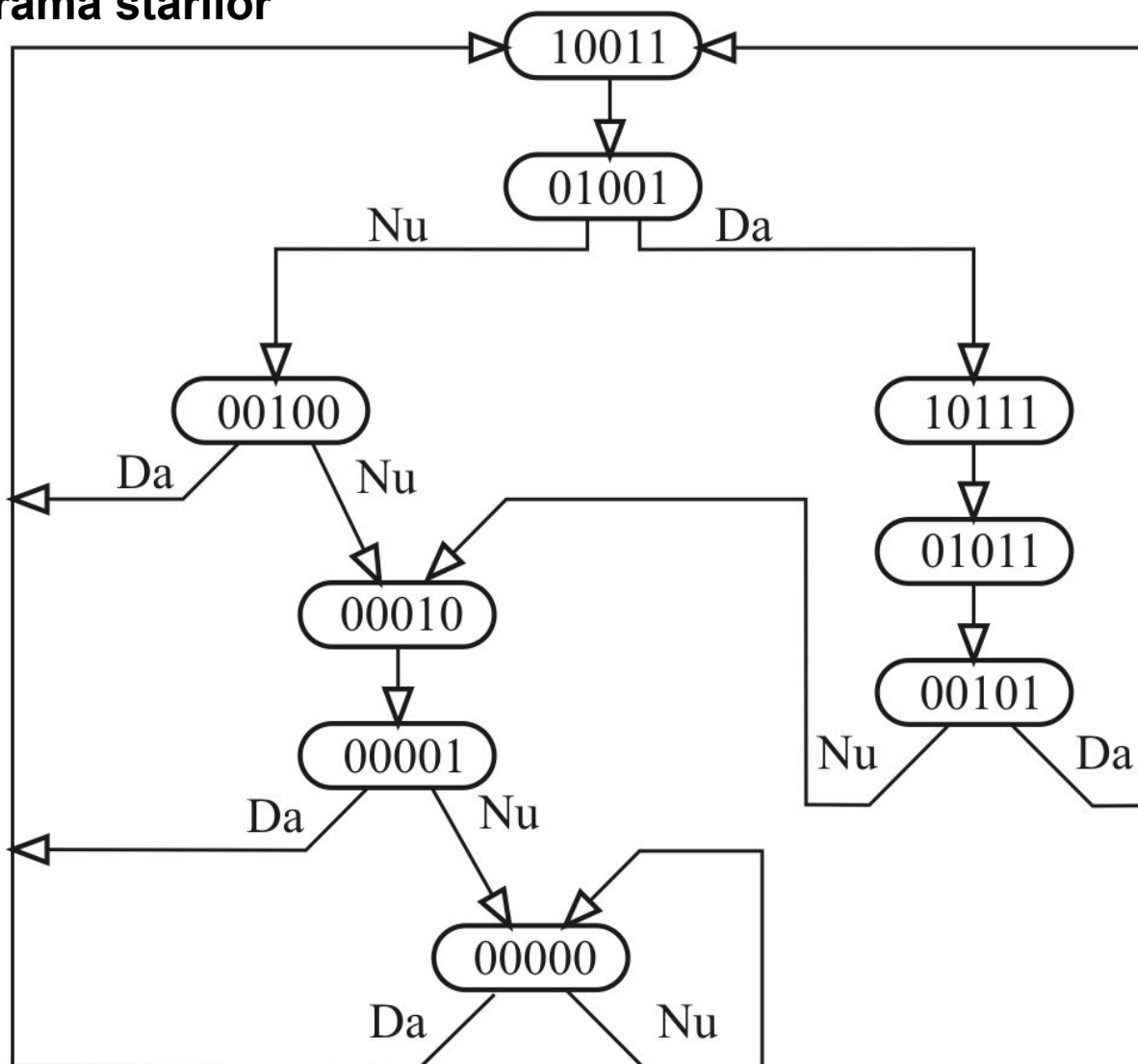
Algoritmul de construcție
a organigramei stărilor



Structuri *pipeline* aritmetice

3. Controlul structurilor statice

C. Organigrama stărilor

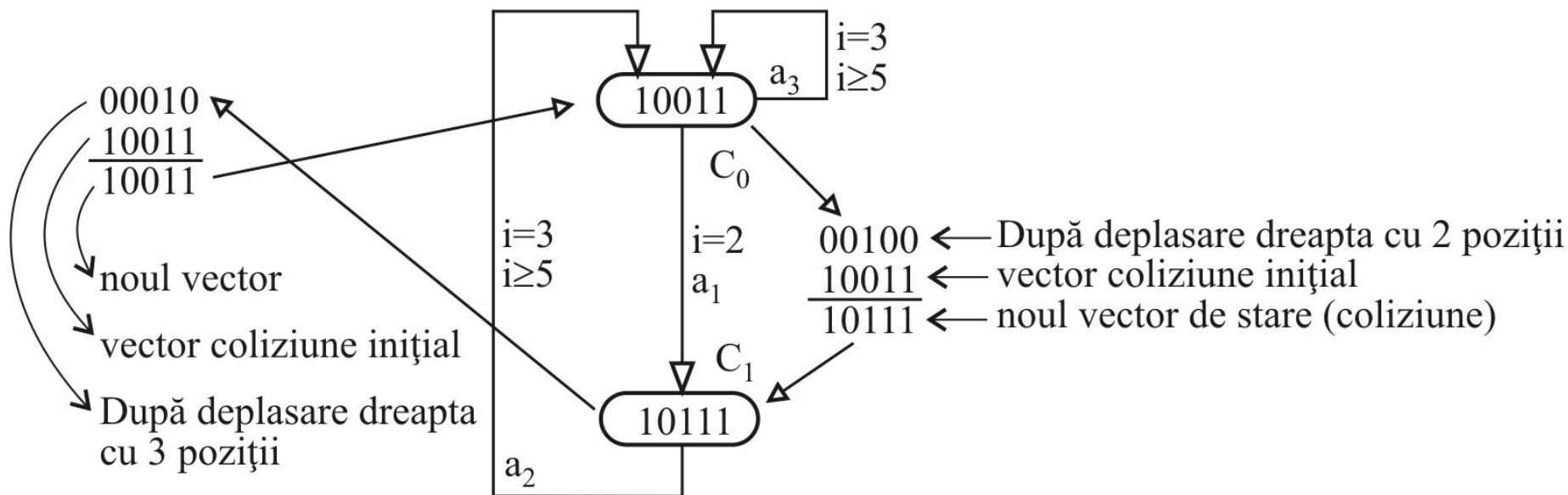


Organigrama stărilor pentru vectorul de coliziune 10011 (DA/NU reprezintă răspunsul la întrebarea "se inițiază un nou *task* ?" din algoritmul de construcție a organigramei)

Structuri *pipeline* aritmetice

3. Controlul structurilor statice

C. Organigrama stărilor



Organigrama simplificată pentru vectorul de coliziune 10011

În organigrama simplificată se reprezintă doar schimbările de stare provocate de inițierea unor noi *task*-uri. Prin urmare, în fiecare stare se iau în considerare doar biții de zero din vectorul de stare; doar aceștia permit inițierea unui nou task

Structuri *pipeline* aritmetice

3. Controlul structurilor statice

D. Latența medie (LM)

Latența medie se determină pentru fiecare ciclu din organigrama stărilor.

În organigrama simplificată de mai sus identificăm două cicluri cu următoarele latențe medii (LM):

$LM = (2+3)/2 = 2,5$ pentru ciclul C_0, a_1, C_1, a_2, C_0 (considerând a_2 cu $i=3$)

$LM = (2+5)/2 = 3,5$ pentru ciclul C_0, a_1, C_1, a_2, C_0 (considerând a_2 cu $i=5$)

$LM = 3/1 = 3$ pentru ciclul C_0, a_3, C_0 (considerând a_3 cu $i=3$)

$LM = 5/1 = 5$ pentru ciclul C_0, a_3, C_0 (considerând a_3 cu $i=5$)

E. Latența medie minimă (LMM)

Fiecare ciclu din organigramă are o latență medie. Ciclul care are cea mai mică latență medie va permite structurii *pipeline* să opereze la performanțe maxime. Această “cea mai mică latență medie” se numește **latență medie minimă** (LMM). Prin urmare, pentru organigrama simplificată de mai sus, obținem:

$$LMM = 2,5$$

aceasta corespunzând ciclului C_0, a_1, C_1, a_2, C_0 (considerând a_2 cu $i=3$).

Asta înseamnă că, în medie după 2,5 tacte (2,5 cicluri *pipeline*), un nou *task* poate fi aplicat pe intrarea structurii *pipeline* (practic: *task*-urile pare după 2 tacte iar cele impare după 3 tacte, rezultând o medie de 2,5 tacte).